11. Exception Handling

11.1 Introduction to Exception Handling

Exception handling in python allows you to handle and mandge runtime errors or exceptional situations that may occur during program execution. Instead of the program abruptly terminating when an a error occurs, exception handling provides a way to catch and handle these errors in controlled manner.

11.2 Exception handling mechanism :

In Python, the exception handling mechanism revolves around the try-except block. The code that might raise an exception is placed within the try block. If an exception occurs, it is caught and handled by the corresponding except block. Multiple except blocks can be used to handle different types of exceptions. There can also be an optional else block, which executes if no exceptions occur, and finally block, which always executes regardless of whether an exception occured or not.

11.3 Handling Multiple Exceptions

Python allows you to handle multiple exceptions using multiple except blocks or a single except block that catches multiple exception types. By specifying

different exception types in seprate except blocks, you can define specific actions to be taken for each type of exception. The order of the except blocks is important beacause the first matching exception block is executed.

11.4 Custom Exceptions

In Python, you can create your own custom exceptions by defining a new class that inherits from the built-in 'Exception' class or its subclasses. Custom Experience exception allows you to define and raise specific types of exceptions that are meaningful for your appln. By creating custom exceptions, you can provide more descriptive error messages and handle error scenarios in more precise way.

11.5 Error Handling Strategies :

In Python, there are several error handling strategies you can employ:

Strategies	Description
Logging	• Use a logging framework, such as the logging module in python, to record and track errors.
Graceful degradation	• Implement fallback mechanisms or alternative paths to handle errors and allow Program for execution

Strategies	Description
Retry mechanism	Implement logic to automatically retry failed operations.
Error reporting	Notify users, system administrators, or relevant stakeholders about errors through appropriate channels.
Graceful termination	Graceful termination ensures data integrity and avoids leaving resources in an inconsistent state.

Date :

12. Advanced Data Structures

List Comprehsion:

Lists comprehsions are a consice way to create lists in python. They allow you to generate lists by applying an expression to each item in an iterable (e.g list, range, or string).

List comprehsions have a compact syntax and are often used for filtering, transforming, or generating lists based on existing data.

The basic structure includes an expression followed by a 'for' loop, and you can optionally include an 'if' clause for conditional filtering.

They are a powerful tool for creating lists efficiently and are considered more readable than traditional 'for' loops in many cases.

Example:

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = [x** 2 for x in numbers if x % 2 == 0]
print (squared_numbers) # [ output : [4, 16]]
```

Example:

```
numbers = [1, 2, 3, 4, 5]
even_numbers = [x for x in numbers if x % 2 == 0]
print (even_numbers) # output : [2, 4]
```

Date : _____

Set comprehsions:

Set comprehsions are similar to list comprehsions but create sets instead of lists.
Sets are collections of unique elements, and set comprehsions automatically remove duplicates.
They are useful for creating sets quickly and efficiently, especially when you want to eliminate duplicate values from an iterable.
sets are unordered collections of unique elements, so set comprehsions automatically remove duplicates.
set comprehsions use curly braces '{}' or the 'set()' constructor.

Example:

```
numbers = [1,2,2,3,3,4,5]
unique_numbers = {x for x in numbers}
print (unique_numbers) #output : {1,2,3,4,5}
```

Dictionary Comprehsions:

In python, dictionaries are the data types in which the users can store the data in a pair of key/value.

Example:

```
dict1 = {"x":12, "r":31, "l":1, "v":54}
```

Dictionary comprehsion is the method used for transferring

Date :

one dictionary into another dictionary. Throughout the process of transferring the dictionary into another, the user can also include the data of the original dictionary into the new dictionary, and each data can be trasffered as per need.

python also allows the user to perform dictionary comprehsions. The user can create the dictionary by using the simple expression of the built-in-method.

The dictionary comprehsion is created in the following form:

{ key: 'value' (value for (key, value) in iterable form)}

Example1:

```
#let's assume the user have two lists named key and values
key = ['p', 'q', 'r', 's', 't']
value = [56, 67, 43, 12, 6]
# the following method is used for comprehensiong the
 dictionary
User_Dict = {X:Y for (X,Y) in zip (key, value)}
print ("user_Dict:", user-Dict)
```

Output:

user_Dict : {'p':56, 'q':67, 'r': 43, 's':12, 't': 6}

The user can also create the dictionary from a list by using comprehension.

Date :

Python stack and Queue (using lists):

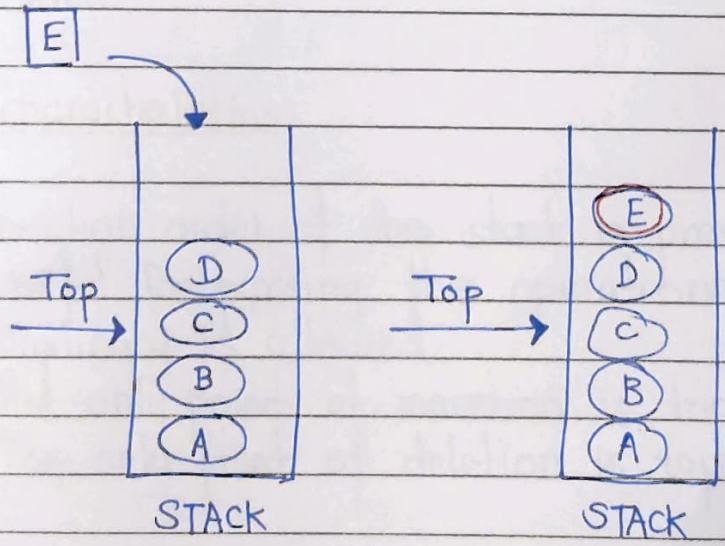
Data structure organizes the storage in computers so that we can easily access the change data. Stacks and Queues are the earliest data structure defined in computer science. A simple python list can act as a queue and stack as well. A queue follows FIFO rule (First In First Out) and used in programming for sorting. It is common for stacks and queues to be implemented with an array or linked list.

Stack:

A stack is a data structure that follows the LIFO (Last In First Out) principle. To implement a stack, we need two simple operations:

push : It adds an element to the top of the stack
pop : It removes an element from the top of the stack.

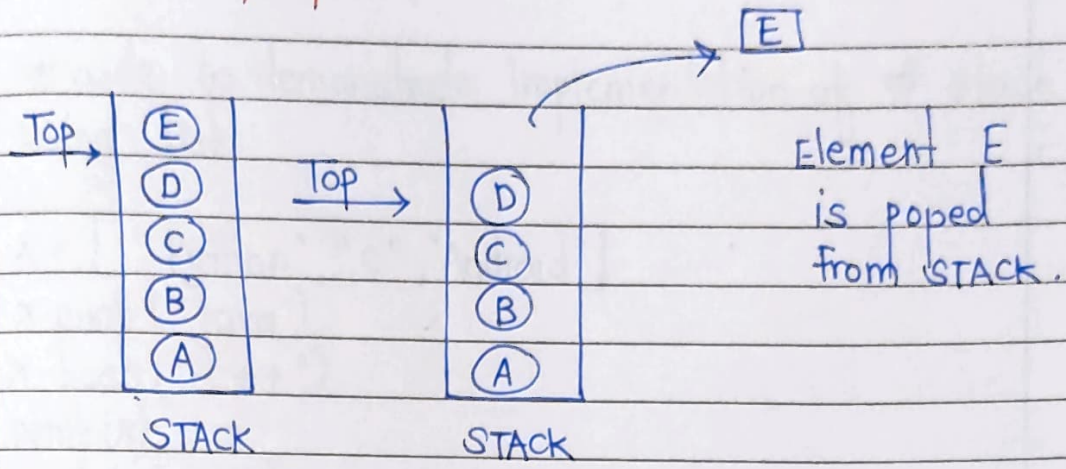
Push operation



Element E is pushed in the stack.

Date :

Pop Operation



Operations:

Adding: It adds the items in the stack and increases the stack size. The addition takes place at the top of the stack.

Deletion: It consists of two conditions, First, if no element is present in the stack, then underflow occurs in the stack, and second, if a stack contains some elements, then the topmost element gets removed. It reduces the stack size.

Traversing: It involves visiting each element of the stack.

characteristics:

1) Insertion order of the stack is preserved.
2) Useful for parsing the operations.
3) Duplicacy is allowed.
4) The only point of insertion is top.
5) The only point of deletion is top.

Date :

Code:

```
# code to demonstrate Implementation of # stack
using list.

x = [ "Python", "c", "Android"]
x.push ("java")
x.push ("c++")
print (x)
print (x.pop())
print (x)
print (x.pop())
print (x)
```

Output:

```
['python', 'c', 'Android', 'Java', 'c++']
c++
['python', 'c', 'Android', 'Java']
Java
['python', 'c', 'Android']
```

Queue :

A queue follows the First-in-First-Out (FIFO) prindple. It is opened from both the ends hence we can easily add elements to the back and can remove elements. The queue has the two ends front and rear . The next element is inserted from the rear end and removed from the front end.

Date :

Operations in python:

Enqueue
Dequeue
Front
Rear

Enqueue: The enqueue is an operation where we add items to the queue. If the queue is full, it is condition of the Queue The time complexity of enqueue is O(1).

Dequeue: The dequeue is an operation where we remove an element from the queue. An element is removed in the same order as it is inserted. If the queue is empty, it is a condition of the Queue Underflow. The time complexity of dequeue is O(1).

Front: An element is inserted in the front end. The time complexity of front is O(1).

Rear: An element is removed from the rear end. The time complexity of rear is O(1).

Date :

13. Functional Programming

Functional programming is a programming paradigm that reads / treats computation as evaluating mathematical functions and avoids changing state and mutable data. In functional programming, functions are first-class citizens, meaning they can be assigned to variable, passed as arguments to other functions, and returned as values from other functions.

In functional programming, functions are first-class citizens, meaning they can be assigned to variables, passed as arguments to other functions, and returned as values from other functions

Functional programming emphasizes immutability, meaning that once a value is assigned, it cannot be changed, and it avoids side affects, which are changes to the state or behaviour that affect the result of a function beyond its return value.

13.1 Map, filter and reduce function :

Map, filter and Reduce are built-in Python functions that can be used for functional programming tasks. With the help of these operations, you may apply a specific function to sequence items using the 'map', filter sequence elements based on a condition using the 'filter', and comulatively aggregate elements using the 'reduce'.

map(): Python's map() method applies a specified function to each item of an iterable (such as a list, tuple,

or string) and then returns a new iterable containing the results.
The map() syntax is as follows: map(function, iterable)

The first argument passed to the map function is itself a function, and the second argument passed is an iterable (sequence of elements) such as a list, tuple, set, string, etc.

Example: usage of the map()

```
data = [1,2,3,4,5]
evens = filter(lambda x: x % 2 == 0, data)
for i in evens:
    print(i, end=" ")
evens = list(filter(lambda x: x % 2 ==0, data))
print(f"Evens = {evens}")
```

Output:

```
2 4
Evens = [2,4]
```

Filter():

The filter() function in Python filters elements from an iterable based on a given condition, or function and returns a new iterable with the filtered elements.

The syntax for the filter() is as follows: filter(function, iterable)

Here, also the first argument passed to the filter function is

Date :

is itself a function, and the second argument passed is an iterable (sequence of elements) such as list, tuple, set, string etc.

Example 1: Usage of the filter():

```
data = [1,2,3,4,5]
evens = filter(lambda x : x%2 == 0,data)
for i in evens:
    print(i, ends=" ")
evens = list (filter (lambda x : x%2 == 0,data))
print(f "Evens = {evens}")
```

Output:

2 4
Evens = [2,4]

reduce() :-

In python, reduce() is a built-in function that applies a given function to the elements of an iterable reducing them to a single value.

The syntax for reduce() is as follows : reduce(function, iterable, [initializer])

The function argument is a function that takes two arguments and returns a single value. The first argument is the accumulated value and the second argument is the current value from the iterable.

Date : _____

The iterable argument is the sequence of values to be reduced.
The optional initializer argument is used to provide an initial value for the accumulated result. If no initializer is specified, the first element of the iterable is used as the initial value.

Example:

```
from functools import reduce
def add(a,b):
    return a+b
num_list = [1,2,3,4,5,6,7,8,9,10]
sum = reduce(add, num_list)
print(f"Sum of the integers of num_list : {sum}")
sum = reduce(add, num_list, 10)
print(f"Sum of the integers of num_list with initial value 10 : {sum}")
```

Output:

```
Sum of the integers of num_list : 55
Sum of the integers of num_list with initial value 10 : 65
```

13.2 Lambda and Anonymous functions:

Lambda functions, also known as anonymous functions, can be defined inline without requiring a formal function declaration and are short and one-time-use functions. They are helpful for the performance of a single task that only requires one line of code to convey

Date :

```
add = lambda x : x+1
multiply = lambda x : x*2
result1 = add(3)
result2 = add(3)
result3 = multiply(3)
```

Method 2 - Using Lambda function:

```
# Using reduce to find the largest of all and printing result
Largest = reduce(lambda x, y : x if x > y else y, num)
print(f"Largest found with method 2 : {largest}")
```

Date :

14. Working with files and Directories

The file handling plays an important role when the data needs to be stored parmanently into the file. A file is a named location on disk to store related information. We can access the stored information (non-volatile) after the program termination.

The file-handling implementation is slightly lengthy or complicated in the other programming language but it is easier and shorter in Python.

In Python, files are treated in two modes as text or binary. The file may be in the text or binary format, and each line of a file is ended with a special character.

Hence, a file operation can be done in the following order:

1) Open a file
2) Read or write - performing operation
3) Close the file

Python Files I/O:

- ## Opening a file

Python provides an open() function that accepts two arguments, file name and access mode in which the file is accessed. The fuction returns a file object which can be used to perform various operations like reading, writing, etc.

Syntax:

file object = open (<file-name>, <access-mode>, <buffering>)

Date : _____

The files can be accessed using various modes like read, write, or append. The following are the details about access mode to open a file.

	Access Mode	Description
1)	r	It opens the file to read-only mode. The pointer exists at the beginning. The file is by default open in this mode if no access mode is passed.
2)	rb	It opens the file to read-only in binary format. The file pointer exists at the beginning of the file.
3)	r+	It opens the file to read and write both. The file pointer exists at the beginning of the file.
4)	w	It opens the file to write only. It overwrites the file if previously exist or creates a new one if no file exist with the same name.
5)	wb	It opens the file to write only in binary format.
6)	w+	It opens the file to write and read both.
7)	wb+	It opens the file to write and read both in binary format.
8)	a	It opens the file in the append mode.
9)	ab	It opens the file in the append mode in binary mode (format)
10)	a+	It opens a file to append and read both.
11)	ab+	It opens a file to append and read both in binary format.

Date :

```
fileptr = open("file.txt","r")
if fileptr:
    print("file is opened")
```

Output:

```
<class – 'io.TextIOWrapper'>
file is opened successfully
```

The close method()

Once all the operations are done on the file, we must close it through our python script using the close() method. Any unwritten information gets destroyed once the close() method is called on a file object.

The syntax to use the close() method is given below:

```
fileobject.close()
```

Consider the example:

```
fileptr = open("file.txt","r")
if fileptr:
    print("file is opened successfully")
fileptr.close()
```

Writing the file

To write some text to a file, we need to open the file using the open method with one of the following access modes:

Date :

w: It will overwrite the file if any file exist. The file pointer is at the beginning of the file.

a: It will append the existing file. The file pointer is at the top-end of the file. It creates a new file if no file exists.

Consider the following example:

```
#open the file.txt in append mode. Create a new file if no such file exists.
fileptr = open("file2.txt", "w")
# appending the content to the file
fileptr.write()
#closing the opened the file
fileptr.close()
```

Output:

```
File2.txt
```

Read file through for loop:

```
#open the file.txt in read mode. causes an error if no such file exists.
fileptr =open ("file2.txt","r");
#running a for loop
for i in fileptr:
    print (i) # i contains each line of the file
```

Output:

```
Python is the modern day language.
```

Date :

14.2 Python os module

Renaming the file

The Python OS module enables interaction with the operating system. The os module provides the functions that are involved in File processing operations like renaming, deleting, etc. It provides us the rename() method to rename the specified file to a new name. The syntax to use the rename() method is given below:

Syntax:

```
rename (current-name, new-name)
```

The first argument is the current file name and the second argument is the modified name. We can change the file name bypassing these two arguments.

Example 1:

```
import os
#rname file2.txt to file3.txt
os.rename ("file2.txt","file3.txt")
```

Removing the file

The os module provides the remove() method which is used to remove the specified file. The syntax to use the remove() method is given below:

Date :

remove (file-name)

Example:

```
import os;
# deleting the file named file3.txt
os.remove("file3.txt")
```

Creating the new directory

The mkdir() method is used to create the directories in the current working directory. The syntax to create the new directory is given below:

mkdir(directory name)

Example

```
import os
# creating a new directory with the name new
os.mkdir("new")
```

The getcwd() method:

This method returns the ~~wo~~ current working directory. The syntax to use the getcwd() method is given below:

Syntax:

```
os.getcwd()
```

Date :

Example

```
import os
os.getcwd()
```

changing the current working directory:

The chdir() method is used to change the current working directory to a specified directory.

The syntax to use the chdir() method is given below.
syntax:

```
chdir("new-directory")
```

Deleting directory

The rmdir() method is used to delete the specific directory
The syntax to use the rmdir() method is given below:

Syntax:

```
os.rmdir(directory name)
```

Example:

```
import os
#removing the new directory
os.rmdir("directory-name")
```

It will remove the specified directory.

14.3 Working with the csv and JSON files

The csv files stands for a comma-seprated values file. It is a type of plain text file where the information is organized in the tabular form. It can contain only the actual text data. The textual data don't need to be seprated by the commas. There are also many seprator characters such as tab (\t), colon (:), and semi-colon (;), which can be used as a seprator.

The csv module work is to handle the csv files to read/write and get data from specified columns. There are different types of csv functions, which are as follows:

csv.field_size_limit: It returns the current maximum field size allowed by the parser.

csv.get_dialect - Returns the dialect associated with a name.

csv.list_dialects - Returns the names of all registered dialects

csv.reader - Read the data from a csv file

csv.writer - Write the data to a csv file

We can also write any new and existing csv files in python by using the csv.writer() module. It is similar to the csv.reader() module and also has two methods. i.e writer function or the Dict Writer class.

It presents two functions i.e writerow() and writerows(). The writerow() function only write one row and the writerows() function write more than one row

Date :

15. Regular Expressions

15.1 Introduction to regular Expression

A regular expression is a set of characters with highly specialized syntax that we can use to find or match other characters or groups of characters. In short, regular expressions, or Regex, are widely used in the UNIX world.

Import the re Module

```
#importing re module
import re
```

The re-module in Python gives full support for regular expressions of pearl style. The re module raises the re.error exception whenever an error occurs while implementing or using a regular expression

Metacharacters or Special Characters

. Dot - It matches any characters except the newline character

^ Caret - It is used to match the pattern from the start of the string (starts with)

$ Dollar - It matches the end of the string before the new line character (Ends with)

* Asterisk - It matches zero or more occurances of a pattern.

+ plus - It is used when we want a pattern to match at least one

? Question mark - It matches zero or one occurence of a pattern.

Date :

[] Bracket - It defines the set of characters
| Pipe - It matches any two defined patterns.

Special Sequences:

Special sequences consists of '\' followed by a character listed below. Each character has a different meaning.

character	Meaning
\d	It matches any digit and is equivalent to [0-9]
\D	It matches any non digit character is equivalent to [^0-9]
\s	It matches any white space character and is equivalent to [\t\n\r\f\v]
\S	It matches any character except the white space character and is equivalent to [^\t\n\r\f\v]
\w	It matches any alphanumeric character and is equivalent to [a-zA-Z0-9]
\W	It matches any characters except the alphanumeric character and is equivalent to [^a-zA-Z0-9]
\A	It matches the defined pattern at the start of the string.
\B	It is opposite of \b
\b	r"\bxt" - It matches the pattern at the beginning of a word in a string
\Z	It returns a match object when the pattern is at the end of the string

RegEx functions:

Date :

compile - It is used to turn a regular pattern into an object of a regular expression that may be used in a number of ways for matching patterns in a string.

search - It is used to find the first occurence of a regrex pattern in a given string.

match - It starts matching the pattern at the beginning of the string.

fullmatch - It is used to match the whole string with regex pattern.

split - It is used to split the pattern based on the regex.

finditer - It returns an iterator that yields match objects

sub - It returns a string after substituting the first occurence of the pattern by the replacement.

escape - It is used to escape special characters in a pattern

purge - It is used to clear the regex expression cache.

Date :

16. Web Development with Python

16.1 Introduction to Web development

Web development involves creating websites and web applications that can be accessed over the internet. These applications can range from simple static websites to complex dynamic web applications that offer various functionalities. Python is a popular programming language for web development due to its simplicity, readability, and a wide range of web frameworks and libraries available for developers.

Components of Web development :

Front-end Development :

Frontend development focuses on creating the user interface and user experience of a website or web application. This involves designing and developing the visual elements that users interact with in their web browsers. Key technologies and concepts in fronted development include:

HTML (Hypertext Markup language) : Used for creating the structure and content of web pages.
CSS (Cascading Style sheet) : Used for styling and layout of web pages.
Javascript : A programming language used for adding interactivity and dynamic behaviour to web pages.
Frontend Frameworks : Libraries and frameworks like react Angular and Vue.Js are often used to streamline front-end development.

Date :

Backend development:

Backend development focuses on the server-side logic and data processing of a web application. Python is commonly used for backend development, and there are several web development frameworks available for this purpose, including

Flask: A lightweight and minimalistic framework for building small to medium-sized web applications

Django: A high-level framework that provides a comprehsive set of tools for building complex web applications.

FastAPI: A modern, fast and asynchronous web framework for building APIs.

Databases:

Most web applications require data storage and storage and retrievel. Python has libraries and frameworks like SQLAlchemy and Django's ORM (Object-Relational Mapping) for interacting with databases, including SQL and NOSQL databases

HTTP and APIs:

The Hypertext Transfer Protocol (HTTP) is the foundation of data communication on the web. Web developers need to understand how HTTP works, as well as how to create and consume APIs (Application Programming Interfaces) to exchange data between the frontend and backend of web application.

Date : _____

Web servers

Python web applications are hosted on web services. Popular web servers for hosting python applications include Gunicorn, uWSGI and ASGI servers like Daphne for asynchronous applications.

Steps in web development:

1. Project Setup:

Choose a web framework (e.g Flask, Django) and set up your development environment by installing python and necessary packages.

2. Design and Layout:

Create the visual design and layout of your web application frontend using HTML and CSS.

3. Backend development:

Write the server-side logic for your web application using your chosen python framework. Define routes, handle HTTP requests and interact with databases.

4. Database Integration:

If your application requires data storage, design the database schema and integrate it with your backend code.

Date :

Testing:

Test your web application throughly to ensure it functions correctly and is free of bugs.

Deployment:

Deploy your web application to a web server or cloud hosting platform, making it accessible to users on the internet.

16.2 Building Simple Web applications with Flask or Django (Basic concepts)

Building simple web applications with Flask or Django involves understanding the basic concepts of these Python web frameworks and how to use them to create a functional web application.

Flask:

Flask is a lightweight and micro web framework for Python. You det It is minimalistic and provides the essential tools for building web applications. Flask is known for its simplicity and flexibility, making it a good choice for small to medium-sized projects.

Basic Concepts:

Routing:

Flask uses routing to map URLs to specific python functions

Date :

You define routes using decorators such as '@app.route("/")', to specify which function should handle a particular URL. For example:

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
  return "Hello, World!"
if __name__ == "__main__":
  app.run
```

Views:

In Flask, views are Python functions that handle requests and return responses. These functions are associated with specific routes. In the above example, the 'home()' function is a view that returns "Hello, World!" when the root URL ("/") is accessed

Templates:

Flask allows you to use templates to seprate the HTML code from Python code. You can use Jinja2, a templating engine, to render dynamic content in HTML templates. This sepration of concerns makes it easier to maintain your code. Example:

```
from flask import Flask, render_template
app = Flask(__name__)
```

Date :

```
@app.route("/")
def home():
    return render_template("index.html", message="Hello, World!")
if __name__ == "__main__"
    app.run()
```

static files:

Flask can serve static files (e.g css, Javascript, images) using the 'static' folder. This helps in organizing and delivering frontend assets efficiently.

Django :

Django is high-level, full-stack web framework for python. It follows the "batteries-included" philosophy, providing many built-in-features for common web development tasks like authentication, database interaction, and form handling.

Basic Concepts :

Project and App structure:

In Django, a project is the entire web application, while an app is a modular component within the project. Django projects can contain multiple apps. The project structure is created for you start a new Django project using the 'django-admin'.

Date : _____

Models:

Django's models define the structure of your database tables. Models are python classes that inherit from 'django.db.models.Model'.

Templates

```
<h1> Hello, {{ user.username }} ! </h1>
```

URL Routing:

Django's uses a URL dispatcher to map URLs to view functions. You define URL patterns in the projects 'urls.py' file.

```
from django.urls import path
from . import views
urlpatterns = [
    path('', views.home, name='home'),
]
```

Date : _____

17. Data Analysis and Visualization

Data analysis is the process of inspecting, cleaning transforming and modeling data to discover useful information, draw conclusions, and support decision-making It's a crucial step in understanding patterns, trends and insights within datasets. Python offers several libraries and tools that are commonly used for data analysis.

Data collection:

Data analysis begins with the collection of relevant data. This data can come from various sources, such as databases, spreadsheets, APIs or web scrapping. Python can be used to automate data retrieval tasks and store data in suitable formats, such as CSV or databases.

Data cleaning:

Raw data often contains inconsistencies, missing values, and errors. Data cleaning involves tasks like: Handling missing data by filling in values or removing rows/columns.

- Correcting data entry errors
- Standardizing data formats
- Python libraries like pandas are widely used for data cleaning due to their flexibility and data manipulation capabilities.

Data Exploration:

Data exploration involves examining the datasets

characteristics and understanding its structure. key techniq
ues include:
- Descriptive statistics to summarize data (e.g mean, medi
, variance)
- Data visualization to create plots and charts for initial insights.
- Exploratory Data Analysis (EDA) to investigate relationships and patterns within the data.
- Libraries like Matplotlib and Seaborn are essential for creating visualizations, while Pandas aids in generating summary statistics

Data Transformation:

- Data may need to be transformed to make it suitable for analysis. Transformations can include:
- Feature engineering to create new variables or extract meaningful information from existing ones.
- Normalization or scaling of data to ensure consistent units and ranges.
- Encoding categorical variables into numerical formats
- Python libraries like Scikit-learn and Pandas provide tools for data transformation.

Data Modeling:

- Once the data is cleared and prepared, modeling techniques can be applied to uncover patterns or make predictions
Common tasks include:
- Regression analysis to understand relationships between variables.

Date : _____

- classification for categorizing data into classes
- clustering to group similar data points
- Time series analysis for temporal data.
- Python offers extensive libraries for machine learning and statistical modeling, including Scikit-learn, stats models and TensorFlow.

Data Interpretation:

- After building models, it's important to interpret the results and draw meaningful conclusions. This can involve
- Analyzing model coefficients or feature importances.
- Assessing model performance through metrics like accuracy, precision, and recall.
- Visualizing model predictions and comparing them to actual data
- Jupyter Notebooks, which integrate code, visualizations and explainatory text, are a popular choice for documenting and sharing data analysis.

Reporting and Visualization:

- Effective communication of findings is essential. Data analysts often create reports and visualizations to convey insights to stakeholders. Python libraries like Matplotlib, seaborn and tools like Jupyter Notebooks facilitate the creation of informative reports.

Automation and Scaling:

- For large datasets or repetitive tasks, automation is

Date :

crucial. Python's scripting capabilities and libraries for data analysis and manipulation enable automation of data processing pipelines and workflows.

17.1 NumPy and Pandas for Data Manipulation

NumPy and Pandas are two fundamental Python libraries commonly used in data analysis and scientific computing. They provide essential tools and data structures for working with numerical data, performing data analysis.

Numpy (Numerical Python):

NumPy is a Python library for numerical and array-based operations. It provides support for multi-dimensional arrays and matrices, along with a vast collection of mathematical functions to operate on these arrays efficiently.

key features and Use Cases:

Arrays:

NumPy's primary data structure is the 'ndarray' (n-dimensiondl array). These arrays are homogeneous and can store elements of the same data type, making them efficient for numerical computations.

Element-Wise Operations:

NumPy allows you to perform element-wise operations on arrays, making mathematical computations more straightforward and efficient.

Date :

Array slicing and indexing:

You can access and manipulate specific elements or subsets of arrays using indexing and slicing.

Broadcasting:

NumPy enables operations on arrays with different shapes, automatically aligning dimensions when possible.

Mathematical Functions:

NumPy Provides a wide range of mathematical functions, including basic arithmetic, linear algebra, statistics and more.

Integration with Other libraries:

NumPy serves as a foundational library for many other scientific computing libraries in Python, such as Scipy and Scikit-learn.

Example of NumPy Usage:

python

```
import numpy as np
#creating NumPy arrays
a = np.array([1,2,3,4,5])
b = np.array([5,6,7,8,9])
#Element-wise addition
```

Date :

```
result = a+b
# computing the mean of an array
mean = np.mean(a)
```

Array Creation:

```
import numpy as np
# Creating a NumPy array
data = np.array([1,2,3,4,5])
```

Element-wise Operations:

```
#Element-wise addition
result = data+2
```

Array slicing:

```
#slicing an array
sub_array = data[1:4]
```

Broadcasting

```
# Broadcasting : Adding a scalar to an array
  data = np.array([1,2,3])
  result = data + 2
  # result : [3,4,5]
```

All above NumPy excels at numerical computations and array operations.

Date :

Pandas:

Pandas is a powerful Python library for data manipulation and analysis. It introduces two primary data structures:
Series (for one-dimensional data) and Dataframe (for two-dimensional, tabular data)

key Features and Use cases:

DataFrame:

The Pandas Dataframe is a versatile data structure that resembles a table or spreadsheet. It can store data of various types, including numerical categorical, and text data.

```
import pandas as pd
data = {'Name': ['Alice', 'Bob', 'charlie'],
        'Age' : [25,30,35]}
    df = pd.DataFrame(data)
```

Data cleaning:

Pandas provides functions for handling missing data, duplicate data, and data type conversions, making data cleaning and preparation more manageble.

```
df.dropna()
df.drop_duplicates()
```

Data selection and filtering:

You can easily select specific rows and columns from a DataFrame based on conditiona, labels or

positions.

```
filtered_data = df[df['Age'] > 30]
```

Merging and Joining:

Data from different sources can be merged or joined together using SQL-like operations.

```
merged_df = pd.merge(df1, df2, on='common_column')
```

Time Series Analysis:

Pandas has excellent support for time series data, including date and time handling, resampling, and rolling calculations.

Aggregation and Grouping:

Pandas allows you to perform aggregation operations on grouped data.

```
df.groupby('category')['value'].mean()
```

Matplotlib:

Data Visualization with Matplotlib:

Basic Plot Types:

Matplotlib supports various plot types, including line plots, scatter plots, bar charts, histograms and pie charts.

Date :

```
import matplotlib.pyplot as plt

# creating a simple line plot

x = [1,2,3,4,5]
y = [10,12,5,8,9]
plt.plot(x,y)
plt.xlabel('x-axis')
plt.ylabel('Y-axis')
plt.title('simple Line Plot')
plt.show()
```

Customization:

You can customize every aspect of a plot, such as labels, colors, legends, titles, and axis properties.

Subplots:

Matplotlib allow you to create multiple plots within a single figure, which is useful for visualizing multiple datasets or comparisons.

Seaborn:

Data Visualization with Seaborn:

statistical plots:

Seaborn simplifies the creation of complex statistical plots. For example: 'sns.scatterplot', 'sns.barplot', 'sns.boxplot', and 'sns.pairplot'

Date :

are designed to provide insights and into data distributions and relationships.

```
import seaborn as sns

sns.set(style = 'whitegrid')
tips = sns.load_dataset('tips')
        sns.lmplot(x='total_bill', y='tip', data=tips
plt.title('scatter plot with Regression Line')
plt.show()
```

Color Palettes:

Seaborn provides a wide range of color pallettes that make your plots visually appealing.

Integration with Pandas:

Seaborn seamlessly integrates with Pandas Dataframes, making it easy to visualize your data directly from Dataframes.

Date :

18. Machine Learning and AI

18.1 Introduction to Machine Learning

Machine Learning (ML) is a branch of artificial intelligence (AI) that focuses on developing algorithms and models capable of learning from data and making predictions or decisions without being explicitly programmed.

Learning from Data:

ML algorithms learn patterns and relationships from historical data. The more data they have, the better they can generalize and make predictions on new, unseen data.

Types of Machine learning:

1) Reinforcement Learning
2) Supervised Learning
3) Unsupervised Learning

Reinforcement Learning:

In reinforcement learning, an agent learns to make sequences of decisions in an environment to maximize a reward signal. Its commonly used in game playing, robotics, and autonomous systems.

Supervised Learning:

Date :

In Supervised learning, the algorithm is trained on labeled data, where the input data is paired with corresponding target labels. It learns to map inputs to output, making it suitable for tasks like classification and the regression.

Unsupervised Learning :

Unsupervised learning deals with unlabeled data. It aims to discover hidden patterns or structures in the data often through techniques like clustering or dimensionally reduction.

Applications of Machine learning :

ML has a broad range of applications, including :

- Image and speech recognition
- Natural language processing (e.g chatbots and language translation)
- Recommender systems (e.g movie recommendation on NetFlix)
- Predictive analytics (e.g stock price forecasting)
- Healthcare diagnostics (e.g identifying diseases from medical images)

18.2 Using Libraries like scikit-learn and Tensorflow):

Machine learning libraries provide tools and frameworks to simplify the development of ML models.

Date :

Two widely used libraries are scikit-learn and TensorFlow

Scikit-learn:

This Python library is an excellent choice for traditional machine learning tasks. It offers a vast collection of tools for data preprocessing, feature selection, model selection, and evalulation. Scikit-learn supports various ML algorithms, making it accessible for both beginners and experienced data scientists.

Data Preprocessing:

Scikit-learn allows you to clean, preprocess, and transform your data easily. For example, you can scale features to have zero mean and unit variance using the 'StandardScaler'.

```
from sklearn.preprocessing import standardScaler
scaler = standardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

Machine Learning Models:

It includes a variety of machine learning algorithms like decision trees, support vector machines, and more. For instance, you can create a simple decision tree classifier:

```
from sklearn.tree import DecisionTreeClassifier
clf = DecisionTreeClassifier()
```

Date :

```
clf.fit(X_train, y_train)
```

Model Evaluation:

Scikit-learn provides tools to evaluate the performance of your models. You can calculate metrics like accuracy, precision, recall, and F1-score:

```
from sklearn.metrics import accuracy_score
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
```

TensorFlow:

TensorFlow is an open-source machine learning framework developed by Google. It specializes in deep learning and neural networks. TensorFlow offers both high-level APIs for quick model development (e.g keras) and lower-level APIs for more fine grained control.

Neural Networks:

TensorFlow is known for its neural network capabilities. You can create complex neural network architectures like convolutional neural networks (CNNs) and recurrent neural networks (RNNs).

```
import tensorflow as tf

model = tf.keras.Sequential([tf.keras.layers.Dense
(128, activation='relu', input_shape=(784,)),
```

Date :

tf.keras.layers.Dropout(0.2),
tf.keras.layers.Dense(10, activation = 'softmax')])

Training:

TensorFlow provides tools for taining neural networks using various optimization algorithms, loss functions and custom training loops.

Deployment:

It supports deployment to various platforms, including mobile devices and the web.

```
import tensorflow as tf from tensorflow.keras
import layers, datasets

# Load the CIFAR-10 dataset (x_train, y_train),

(x_test, y_test) = datasets.cifar10.load_data()
x_train, x_test = x_train/255.0, xtest/255.0

model = tf.keras.Sequential([layers.Conv2D(32, (3,3),
        activation ='relu'), layers.Dense(10)])

# compile the model

model.compile(optimizer ='adam', loss = tf.keras.losses
.SparseCategoricalCrosee ntrophy (from_logits = True),

metrics = ['accuracy'])
```

Date : _____

```
model.fit(x_train, y_train, epochs = 10, validation_data
= (x_test, y_test))
```

18.3 Simple Machine learning Example:

Prediction Iris Flower Species:

Step1 Data Preparation:

- Dataset : We'll use the famous Iris dataset, which contains measurements of special length, sepal width, petal length and petal width for three species of iris Flowers : setosa, versicolor, and verginica.
- Data cleaning : Ensure there are no missing values or outlie
- Data spliting : Divide the dataset into two parts - a training set and a testing set. Typically, a common split is 80% for training and 20% for testing.

step2 : choose an algorithm:

In this case, we'll use a simple classification algorithm, such as a decision tree classifier.

Step 3: Training the Model.

Feed the training data (features like sepal length, sepal width, petal length, and petal width) and their corresponding labels (iris species) into the decision tree classifier. The algorithm will learn the patterns and relationships between the features and the species during training

Date :

step 4 : Evaluation:

- Use the testing dataset to access the model's performance.
- Calculate metrics like accuracy, precission, recall, and F1-score to understand how tell the model is classifying iris Flowers.

step 5 Making predictions:

- Once the model is trained and evaluated, it can be used to predict the species of iris flowers when given new medsurements.

Step 6 Fine tuning:

- Adujust hyperparameters of the decision tree, like the maximum depth or minimum samples per leaf, to improve the model's performance.
- Consider tying different algorithms or ensemble methods (e.g Random Forest) to see if they yeild better results.

Here's a simplified python code snippet using scikit-learn to perform this task.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
From sklearn.metrics import accuracy_score

#Load the Iris dataset
```

Date :

```
iris = load_iris()
x = iris.data
y = iris.target

#split the data into training and testing sets.

x_train, x_test, y_train, y_test = train_test_split(x,y, test_size
  =0.2, random_state=42)

# Create a Decision Tree classifier.

clf = DecisionTreeclassifier()

# Train the model on the training data.

clf.fit(x_train, y_train)

#Make predictions on the testing data
y_pred = clf.predict(x_test)

#Evaluate the model's accuracy

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```